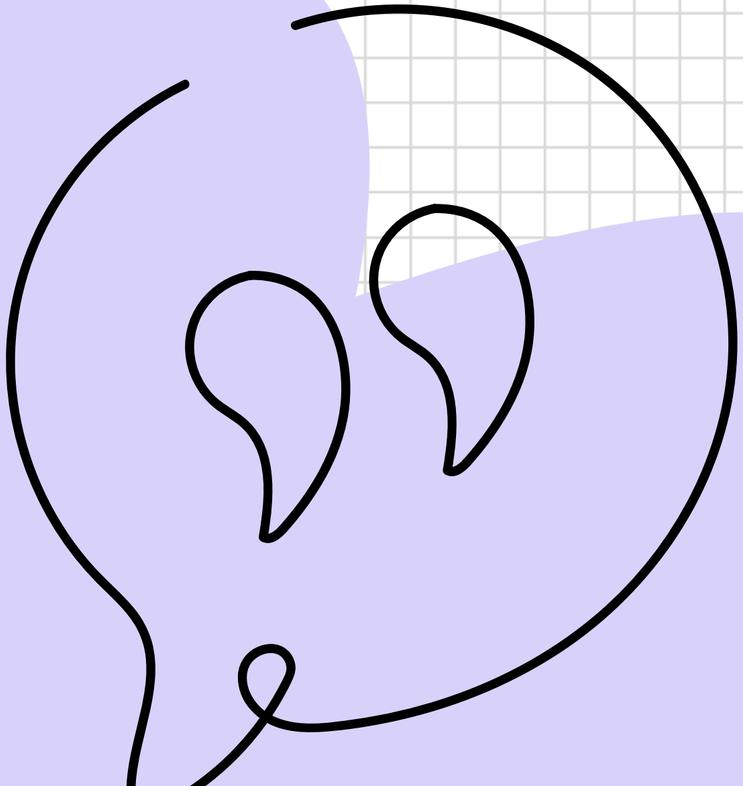
*POINTERS PE CHARCHA

CONTENTS



STRUCTURE POINTERS – DEFINITION

A structure pointer is a pointer variable that stores the address of a structure, the pointer is used to access and manipulate the data stored in the structure.

Structure pointers are commonly used when structures contain large amounts of data or when dynamic memory allocation is required.

STRUCTURE POINTERS – SYNTAX AND USAGE

Basic Syntax:
struct Student *ptr;

ptr = &studentVariable;

Members of the structure are accessed using the arrow (→) operator.

Structure pointers improve efficiency and enable array of pointers to structures.

STRUCTURE POINTERS – CHHOTU SA EXAMPLE

```c
// Structure pointers:
// Q1: Find error/output
#include<stdio.h>
struct S {
    int x;

};

int main() {
    struct S s = {45};
    struct S *p = &s;

    printf("%d", *(p.x));
    return 0;
}
// Compilation error
// . has higher precedence than *
//Here we must use -> operator
```

```c
// Compilation error
// . has higher precedence than *
//Here we must use -> operator
```

```
questions.c: In function 'main':
questions.c:13:27: error: 'p' is a pointer;
 did you mean to use '->'?
     printf("\n%s\n\n", *(p.msg));
                           ^
                           ->
PS C:\Users\nisar\.vscode\vscode\PPC>
```

COMPUTER SOCIETY OF INDIA ESTD. 1965 सर्वे भवन्तु सुखिनः

STRUCTURE POINTERS – PREVIOUS YEAR QUESTION

- **Develop a complete C program to maintain the records of students such as name and marks of five subjects.**

- **Calculate total marks of each student and display the student records in alphabetical order of their names using array of pointers to structures. (Student_Result.c)**

POINTERS PASSED AS PARAMETERS - DEFINITION

- When pointers are passed as arguments to a function, the function receives the address of the actual variable.
- This technique allows the function to modify the original data directly and is commonly known as call by address.

POINTERS PASSED AS PARAMETERS – SYNTAX AND BENEFITS

Basic Syntax:

- **void functionName(int *ptr);**
- **Passing pointers avoids copying data.**
- **It allows modification of original variables.**
- **It is efficient for handling arrays and structures.**

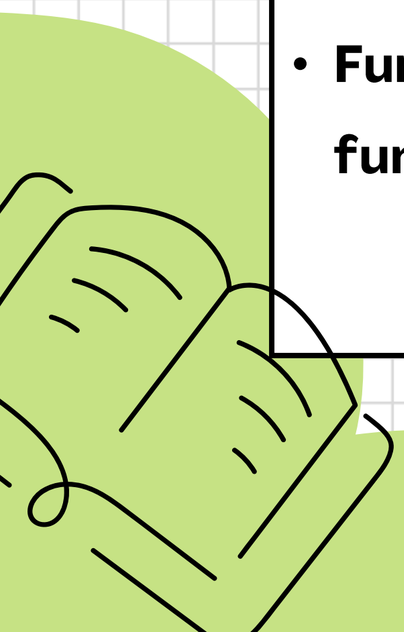
POINTERS PASSED AS PARAMETERS – PREVIOUS YEAR QUESTION

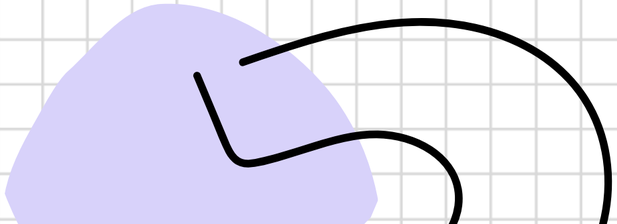
- **Explain the concept of call by address in C programming and write a program to swap two numbers using a function by passing pointers as arguments and explain its working.**

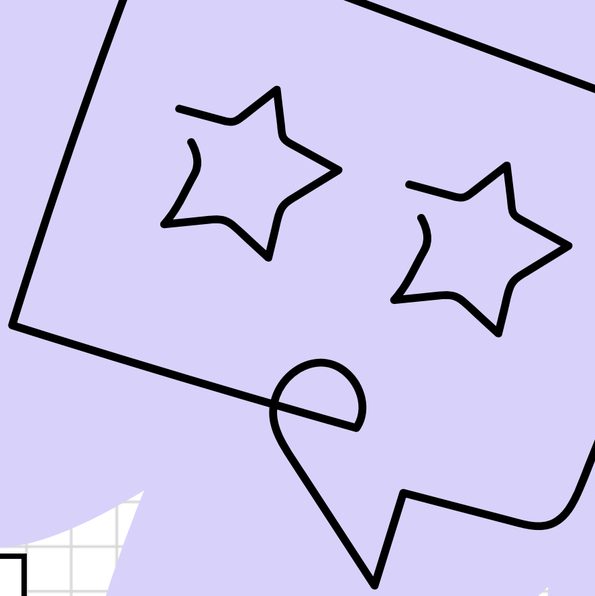
(Function_Parameters.c)¡

FUNCTION POINTERS – DEFINITION

- A function pointer is a pointer that stores the address of a function instead of a variable.
- Function pointers enable dynamic selection and execution of functions during runtime.







FUNCTION POINTERS – SYNTAX AND APPLICATIONS

Basic Syntax :

- **return_type (*pointer_name)(parameter_list);**
- **Function pointers are used in callbacks.**
- **They are useful in menu driven programs.**
- **They improve flexibility and modularity.**

FUNCTION POINTERS – PREVIOUS YEAR QUESTION

- **Interpret the following declaration and explain its meaning clearly:**

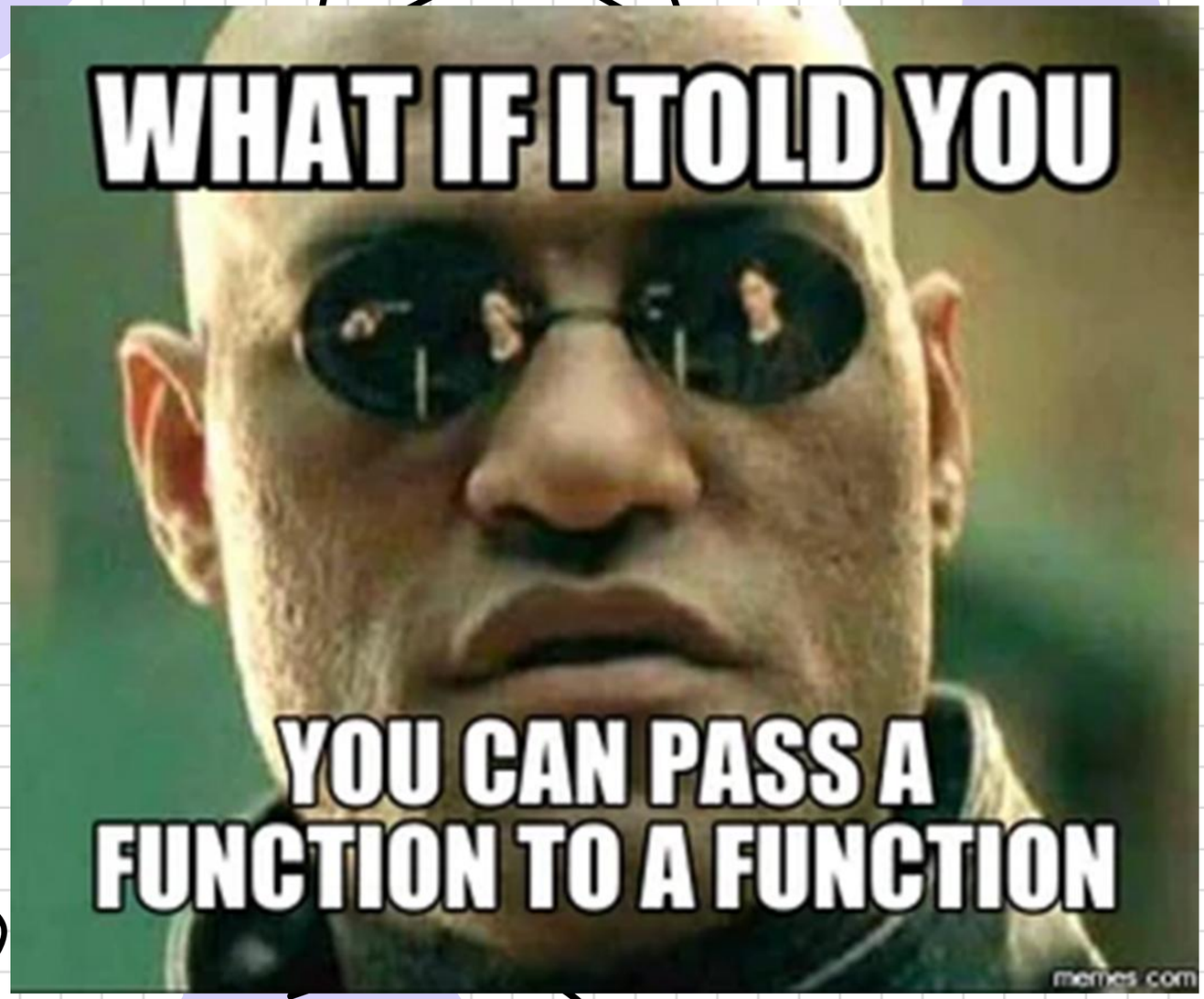
- **Question : int *(*p[4])(int*, int*);**

•p[4]: p is an array of 4 elements.

•*p[4]: The elements of this array are pointers.

•(*p[4])(int*, int*): They point to functions that take two int* arguments.

•int *...: Those functions return a pointer to an int.


WHAT IF I TOLD YOU
YOU CAN PASS A FUNCTION TO A FUNCTION
memes.com

USE CASE - CALLBACKS

- **Functions are passed as arguments to other functions. This enables flexible, reusable code patterns.**
- **CallBack example :**

```
void execute(int (*operation)(int, int), int x, int y) {
    printf("Result: %d\n", operation(x, y));
}


execute(add, 5, 3);  // Passes 'add' function
// Output: Result: 8
```

Callback Functions


"Let's finish setting up"
OneDrive
Me wanting to
keep my files offline,
safe, and organized

FILE HANDLING IN C – DEFINITION

- File handling in C refers to the process of creating, opening, reading, writingandclosing files stored locally.
- It enables programs to store data permanently and retrieve it later.

FILE HANDLING – FILE POINTER AND OPEN MODES

- File operations in C are performed using a FILE pointer.
- Syntax: FILE *fp;
- fopen() is used to open a file.
- Modes include r, w, a, r+, w+, a+.

FILE HANDLING – READING AND WRITING FUNCTIONS

fprintf() is used to write formatted data to a file.

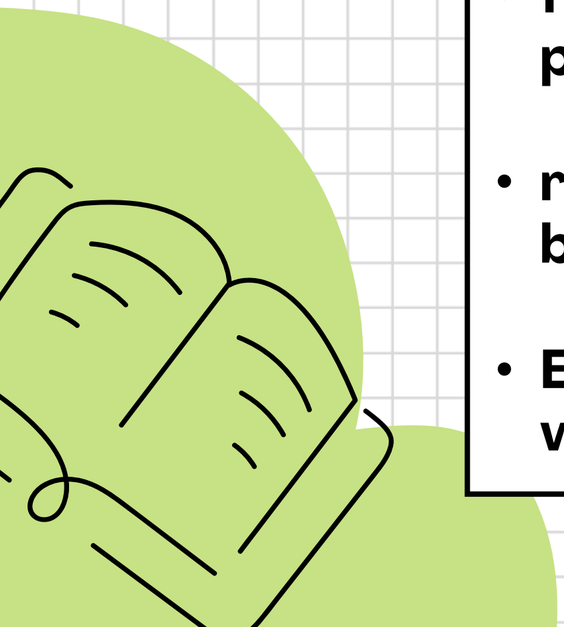
fscanf() is used to read formatted data from a file.

putc() and getc() are used for character based file operations.

fgets() is used to read a complete line from a file.

FILE HANDLING – FILE NAVIGATION AND EOF

- fseek() is used to move the file pointer to a specific location.
- ftell() returns the current position of the file pointer.
- rewind() moves the file pointer back to the beginning.
- EOF indicates end of file and feof() checks whether EOF has been reached.



FILE HANDLING – PREVIOUS YEAR QUESTION

Explain file handling in C programming.

- **Describe various file handling functions such as fopen(), fclose(), fprintf(), fscanf(), fseek(), rewind(), ftell() and EOF with suitable examples.**

(**File_Handeling.c**)

GOT IT !


WHEN YOU FINALLY UNDERSTAND POINTERS
Sometimes it's hard being superior to every single other person on the planet.

MALLOC() MEMORY ALLOCATION IN C

Theory:

- malloc() dynamically allocates a single block of memory from the heap at runtime. The allocated memory is not initialized and may contain garbage values.

Syntax:

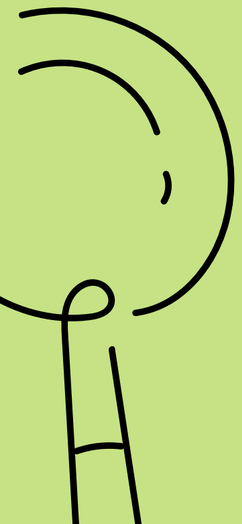
- ptr = (datatype *) malloc(size_in_bytes);

Example:

- int *ptr;
- ptr = (int *) malloc(5 * sizeof(int));

Key Points:

- Faster than calloc()
- Manual initialization required
- Returns NULL if allocation fails



CALLOC() CONTIGUOUS ALLOCATION IN C

Theory:

- **calloc() allocates memory for multiple elements and initializes all allocated memory to zero. It is safer when default values are required.**

Syntax:

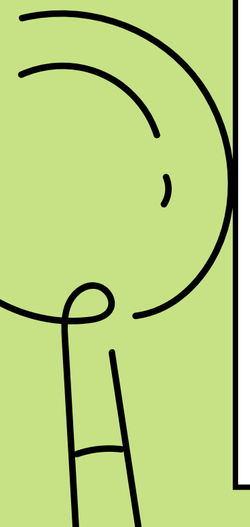
- **ptr = (datatype *) calloc(number_of_elements, size_of_each);**

Example:

- **int *ptr;**
- **ptr = (int *) calloc(5, sizeof(int));**

Key Points:

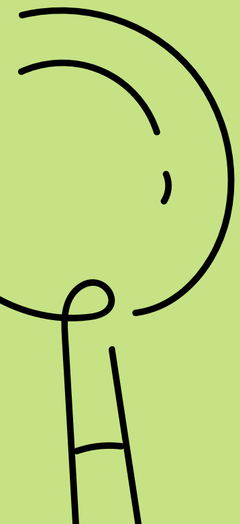
- **Memory initialized to zero**
- **Slightly slower than malloc()**
- **Useful for arrays and structures**



IMPORTANT NOTES

- Both malloc() and calloc() allocate memory from the heap
- Always check for NULL pointer after allocation
- Memory must be freed using free(ptr)
- Failure to free memory leads to memory leaks

Now let us move to VS Code and observe how malloc and calloc behave during execution.
calloc_malloc.c



CSI STUDENT BRANCH, DDU

RIDHI RATHOD	NISARG PATEL	HETVI RAJPUROHIT